

FraudGuard MLOps Platform

Google Cloud Platform — Architecture, Resources & Configuration Reference Guide

Project ID: vizualflo-openclaw-prod | Region: us-central1

Prepared by: Harish Thorath | Date: June 2026

Category	Value
GCP Project ID	vizualflo-openclaw-prod
Primary Region	us-central1
Services Used	GCS · Artifact Registry · Vertex AI · Cloud Run · GKE Autopilot · VPC · IAM · Workload Identity
Terraform State	GCS bucket: fraudguard-tf-state
CI/CD SA	fraudguard-cicd@vizualflo-openclaw-prod.iam.gserviceaccount.com
Serving Endpoint	Cloud Run: fraudguard-serve (public, min=0, max=3 instances)
ML Framework	XGBoost + MLflow 3.x + Evidently 0.7 + DVC + KFP / Vertex AI Pipelines

Document Purpose: This reference guide explains every Google Cloud Platform service used in the FraudGuard real-time fraud detection MLOps pipeline — what it is, why it was chosen, how it is configured, and how to explain it to interviewers, stakeholders, or team members. Sections are ordered from foundational to advanced.

Table of Contents

1.	Project Overview & MLOps Architecture	3
2.	Google Cloud Storage (GCS)	4
3.	Artifact Registry	6
4.	Vertex AI Pipelines (KFP)	8
5.	Cloud Run	11
6.	GKE Autopilot	13
7.	VPC Network & Subnetwork	16
8.	IAM & Service Accounts	17
9.	Workload Identity Federation	19
10.	Terraform (Infrastructure-as-Code on GCP)	20
11.	DVC with GCS Remote	22
12.	GitHub Actions + GCP Integration	23
13.	ArgoCD + GKE GitOps	25
14.	Full GCP Resource Inventory Table	26
15.	How to Explain This Project (Interview Script)	27

1. Project Overview & MLOps Architecture

FraudGuard is a production-grade, real-time credit-card fraud detection platform built as a complete MLOps pipeline. It covers the entire machine-learning lifecycle: data versioning, model training, experiment tracking, model registry, online serving, drift monitoring, automated retraining, and GitOps-based Kubernetes deployment — all powered by Google Cloud Platform.

The Business Problem

Credit-card fraud is rare (roughly 1 in 100 transactions) but costly. A static model trained once degrades as fraudsters change tactics — a phenomenon called *data drift*. FraudGuard solves this with an always-on pipeline that detects drift automatically and retrains the model without human intervention.

End-to-End MLOps Loop

Step	Component	GCP Service
1. Ingest & Version	DVC tracks raw CSV datasets	GCS (dvc remote)
2. Train	XGBoost with scale_pos_weight=99 for 99:1 class imbalance	Vertex AI Pipeline / local GPU
3. Track	MLflow logs params, metrics, artifacts	GCS (mlflow artifacts)
4. Register	MLflow Model Registry, alias @production	GCS (model artifacts)
5. Serve	FastAPI, Prometheus metrics, /predict endpoint	Cloud Run (serverless)
6. Monitor	Evidently drift report on inference data	GCS (inference logs)
7. Alert	Drift exit-code 1 triggers GitHub Actions	—
8. Retrain	Automated retrain + promote pipeline	Vertex AI Pipelines
9. Deploy	Docker image pushed, GKE canary rollout via ArgoCD	Artifact Registry + GKE Autopilot

Key Numbers

Dataset: 100,000 synthetic transactions 1% fraud rate 10 features
Model: XGBoost val PR-AUC = 0.9901 test PR-AUC = 0.9857 ROC-AUC = 0.9997 F1 = 0.9598
Drift threshold: >30% of features drifted triggers retraining
Serving SLO: p99 latency <100ms error rate <1%

2. Google Cloud Storage (GCS)

What is Google Cloud Storage?

Google Cloud Storage is GCP's object storage service — similar to Amazon S3. It stores any amount of unstructured data (files, CSV datasets, model binaries, JSON reports) in *buckets*. Data is globally durable (99.999999999% durability), strongly consistent, and accessible via REST API or the gsutil / gcloud CLI.

Buckets Used in FraudGuard

Bucket 1 — Main Data Bucket

Bucket Name	vizualflo-openclaw-prod-fraudguard-data
Location	us-central1 (single-region, low latency)
Storage Class	STANDARD (frequent access)
Versioning	Enabled — keeps all previous versions of every object
Uniform Access	Enabled — IAM only, no legacy ACLs
Lifecycle Rule	Delete objects older than 90 days (controls storage cost)
Terraform File	infra/terraform/main.tf — resource google_storage_bucket

Folder Structure Inside the Bucket

gs://vizualflo-openclaw-prod-fraudguard-data/
data/transactions.csv — raw training dataset (DVC-tracked)
data/transactions_drifted.csv — drifted version for drift simulation
reference/reference.csv — Evidently baseline (training distribution)
dvc/ — DVC content-addressed cache (SHA-256 hashed)
pipelines/ — Vertex AI pipeline root (component I/O)
mlflow-artifacts/ — MLflow run artifacts (model.xgb, metrics)
inference/latest.csv — live prediction logs for drift monitoring

Bucket 2 — Terraform State Bucket

Bucket Name	fraudguard-tf-state
Purpose	Stores terraform.tfstate — single source of truth for all GCP resources
Access	Only the CI/CD service account can read/write

Backend Block

```
infra/terraform/main.tf — terraform { backend "gcs" { bucket = "fraudguard-tf-state"
prefix = "terraform/state" } }
```

Why GCS?

- **Central truth for data versioning:** DVC (Data Version Control) uses GCS as its remote cache. Every dataset version is stored as a SHA-256-named blob. Team members pull the exact same data with dvc pull.
- **MLflow artifact store:** When MLflow logs a trained model, it uploads model.xgb, MLmodel, and conda.yaml to GCS. Any downstream service can load the model without copying files manually.
- **Vertex AI pipeline root:** Vertex AI Pipelines stores component inputs/outputs (feature matrices, evaluation metrics JSON) as GCS objects. Components communicate by writing and reading from the pipeline root folder.
- **Inference logging:** The serving endpoint writes prediction logs to GCS. The drift detector reads these logs and compares them to the reference distribution.
- **Terraform state:** Terraform uses GCS as a remote backend so that infrastructure state is shared across the team and CI/CD pipelines — no one person holds the state locally.

How to Explain GCS in an Interview

"GCS is our persistent store. Think of it as the filesystem of the platform. Training data, versioned by DVC, lives at gs://...fraudguard-data/data/. The trained model artifacts live at mlflow-artifacts/. When Vertex AI runs a pipeline step, inputs and outputs flow through the pipelines/ prefix. Versioning is on so we can roll back to any dataset snapshot. A lifecycle rule auto-deletes objects older than 90 days to keep costs down."

3. Artifact Registry

What is Artifact Registry?

Artifact Registry is GCP's fully managed repository for storing and serving container images (Docker) and language packages (Maven, npm, PyPI). It replaces the older Container Registry (gcr.io). It integrates natively with Cloud Run, GKE, Vertex AI, and Cloud Build for authenticated image pulls without extra configuration.

Repository Configuration

Repository Name	fraudguard
Format	DOCKER
Region	us-central1
Full Registry URL	us-central1-docker.pkg.dev/vizualflo-openclaw-prod/fraudguard
Terraform File	infra/terraform/main.tf — resource google_artifact_registry_repository
Description	FraudGuard Docker images

Images Stored

Image	Tag Pattern	Built By	Used By
fraudguard-serve	latest, run-{N}	GitHub Actions CI/CD	Cloud Run, GKE

Docker Image Contents

The image is built from docker/Dockerfile.serve and packages:

- FastAPI application (src/serve/app.py)
- XGBoost, MLflow client, Prometheus client
- Python 3.11 slim base image
- Exposes port 8000 | CMD: uvicorn src.serve.app:app --host 0.0.0.0 --port 8000

CI/CD Push Workflow

1. GitHub Actions authenticates to GCP using google-github-actions/auth@v2 (Workload Identity)
2. Runs: docker build -t us-central1-docker.pkg.dev/vizualflo-openclaw-prod/fraudguard/fraudguard-serve:{run_number} .
3. Runs: docker push us-central1-docker.pkg.dev/.../fraudguard-serve:{run_number}
4. Also tags and pushes :latest for Cloud Run default pull
5. Cloud Run / GKE pulls the image on next deploy using the Artifact Registry URL

Why Artifact Registry instead of Docker Hub?

- **Private by default:** Images are not publicly accessible; only IAM-authorized identities can pull.
- **No rate limits:** Docker Hub throttles anonymous pulls (100/6h). Artifact Registry has no pull limit.
- **Same-network pulls:** GKE and Cloud Run pull images from within GCP, so no egress cost and lower latency.
- **Integrated vulnerability scanning:** Artifact Registry can scan images for known CVEs automatically.
- **Native IAM:** The CI/CD service account is granted roles/artifactregistry.writer — only it can push; GKE nodes can only pull.

How to Explain Artifact Registry in an Interview

"Artifact Registry is our private Docker image store. Every time GitHub Actions retrains and rebuilds the serving container, it pushes the new image to us-central1-docker.pkg.dev/.../fraudguard-serve tagged with the CI run number. Cloud Run and GKE always pull from this registry — no public Docker Hub, so images are private and scanned for vulnerabilities. The service account has artifactregistry.writer so only CI can push; the GKE nodes only need reader."

4. Vertex AI Pipelines (KFP)

What is Vertex AI?

Vertex AI is GCP's unified machine learning platform. It provides managed services for training, hyperparameter tuning, serving, feature stores, and pipeline orchestration. In FraudGuard we specifically use **Vertex AI Pipelines** — the managed Kubeflow Pipelines (KFP) runtime. A pipeline is a directed acyclic graph (DAG) of *components*, where each component is a containerised function.

Pipeline Configuration

Pipeline File	src/pipelines/vertex_pipeline.py
Compiled Output	/tmp/fraudguard_pipeline.json
Pipeline Root	gs://vizualflo-openclaw-prod-fraudguard-data/pipelines
Service Account	fraudguard-cicd@vizualflo-openclaw-prod.iam.gserviceaccount.com
Region	us-central1
GCP Console URL	https://console.cloud.google.com/vertex-ai/pipelines?project=vizualflo-openclaw-prod

Pipeline DAG — 4 Components

Step	Component Function	Inputs	Outputs	What It Does
1	validate_data	GCS CSV path	row_count, schema_ok	Downloads CSV from GCS, checks row count >1000, checks all 10 feature columns exist
2	train_model	validated=True, GCS path	model_uri, pr_auc	Loads data, trains XGBoost (scale_pos_weight=99), logs to MLflow, registers model
3	promote_model	model_uri, pr_auc	promoted=True/F else	Checks PR-AUC >=0.85; if yes, sets @production alias via MLflow client
4	deploy_cloud_run	promoted=True, image_uri	(service URL)	Runs gcloud run deploy, sets min=0 max=3, injects env vars, returns serving URL

Component Code Structure

Each component is a Python function decorated with `@component` from `kfp.v2.dsl`. The decorator packages the function into a container, uploads it to Artifact Registry, and KFP runs it as a separate pod in GKE:

```
@component(base_image='python:3.11-slim', packages_to_install=['xgboost','mlflow',...])

def train_model(data_path: str, mlflow_uri: str) -> NamedTuple('Outputs', [('model_uri', str), ('pr_auc', float)]):

# All code runs inside a GKE pod provisioned by Vertex AI
```



```
...  
  
return model_uri, pr_auc
```

Pipeline Submission Code

```
from kfp.v2 import compiler  
  
from google.cloud import aiplatform  
  
compiler.Compiler().compile(pipeline_func=fraudguard_pipeline,  
                             package_path='/tmp/fraudguard_pipeline.json')  
  
aiplatform.init(project='vizualflo-openclaw-prod', location='us-central1')  
  
job = aiplatform.PipelineJob(  
    display_name='fraudguard-pipeline',  
    template_path='/tmp/fraudguard_pipeline.json',  
    pipeline_root='gs://vizualflo-openclaw-prod-fraudguard-data/pipelines',  
    parameter_values={'data_path': 'gs://.../transactions.csv', ...}  
)  
  
job.run(service_account='fraudguard-cicd@...')
```

Why Vertex AI Pipelines?

- **Reproducibility:** Every pipeline run is logged with exact inputs, outputs, component versions, and metrics. You can re-run any historical pipeline.
- **Scalability:** Each component gets its own GKE pod with configurable CPU/GPU/memory. The data validation step can run on a tiny pod; the training step on a GPU node pool — independently.
- **Dependency management:** KFP DAG ensures `validate_data` always runs before `train_model`. If validation fails, downstream steps are skipped automatically.
- **Auditability:** The Vertex AI console shows every run, parameter, metric, and artifact link. Regulators/compliance teams can audit the entire training lineage.
- **Managed infrastructure:** No need to manage Argo Workflows or Airflow. Vertex AI provisions and tears down pods for each run — zero cluster management overhead.
- **Integration with GCS:** Component I/O (DataFrames, model files, reports) flows through GCS automatically. No hand-wiring of file paths between steps.

How to Explain Vertex AI Pipelines in an Interview

"Vertex AI Pipelines is our ML orchestrator. When drift is detected, GitHub Actions triggers the pipeline defined in vertex_pipeline.py. The pipeline has four steps: validate the data from GCS, train the XGBoost model and log it to MLflow, promote to the production alias only if PR-AUC exceeds 0.85, and finally deploy the new image to Cloud Run. Each step runs in its own container on GKE — completely isolated and reproducible. We can inspect every run in the Vertex AI console and trace exactly which data produced which model."

5. Cloud Run

What is Cloud Run?

Cloud Run is GCP's fully managed serverless container platform. You provide a Docker image; Cloud Run handles everything else — HTTPS endpoint, TLS certificate, auto-scaling (including scale-to-zero), health checks, and rolling deployments. It charges only for the CPU and memory consumed during request processing.

Cloud Run Service Configuration

Service Name	fraudguard-serve
Region	us-central1
Container Image	us-central1-docker.pkg.dev/vizualflo-openclaw-prod/fraudguard/fraudguard-serve:latest
Container Port	8000
CPU	1 vCPU
Memory	1 GiB
Min Instances	0 (scales to zero when idle — saves cost)
Max Instances	3 (caps maximum concurrency)
Public Access	Yes — allUsers granted roles/run.invoker
Liveness Probe	GET /health on port 8000
Env Vars	MODEL_NAME=fraudguard-xgboost MODEL_STAGE=production
Terraform File	infra/terraform/cloudrun.tf — resource google_cloud_run_v2_service

Deployment Flow

1. GitHub Actions builds Docker image from docker/Dockerfile.serve
2. Pushes to Artifact Registry: us-central1-docker.pkg.dev/.../fraudguard-serve:latest
3. deploy_cloud_run Vertex AI component runs:
gcloud run deploy fraudguard-serve
--image us-central1-docker.pkg.dev/.../fraudguard-serve:latest
--region us-central1 --project vizualflo-openclaw-prod
--memory 1Gi --cpu 1 --concurrency 80 --allow-unauthenticated
4. Cloud Run performs rolling zero-downtime swap of the container
5. New service URL returned and logged as pipeline output

FastAPI Endpoints Served

Endpoint	Method	Description
POST /predict	POST	Accepts a transaction JSON, runs XGBoost inference, returns fraud_probability, decision (ALLOW/BLOCK), latency_ms
GET /health	GET	Returns {status: ok, model_loaded: true} — used by Cloud Run liveness probe
GET /metrics	GET	Exposes Prometheus metrics: requests_total, latency_histogram, fraud_score_histogram
GET /docs	GET	Swagger UI auto-generated by FastAPI

Why Cloud Run?

- **Zero ops:** No VMs, no Kubernetes cluster management, no load balancer config. Deploy a container and get an HTTPS endpoint in under 60 seconds.
- **Scale-to-zero:** min_instances=0 means the service shuts down when idle. For a portfolio project with low traffic, this keeps the bill at \$0 between demo sessions.
- **Cost efficiency:** Charged per 100ms of actual request processing. If the service handles 1000 predictions/day, cost is fractions of a cent.
- **Canary-ready:** Cloud Run supports traffic splitting. We can send 10% of traffic to a new revision and roll back instantly if error rates rise.
- **Integrated with Artifact Registry:** Cloud Run pulls images directly from GCP's Artifact Registry with no credentials — same-project IAM handles auth.

How to Explain Cloud Run in an Interview

"Cloud Run is our production serving layer. After training, the Vertex AI pipeline calls gcloud run deploy with the new container image from Artifact Registry. Cloud Run gives us an HTTPS endpoint instantly — no load balancer, no certificate management. min_instances=0 means it scales to zero overnight, so we are not paying for idle VMs. The FastAPI /predict endpoint runs inside the container and returns a fraud score in under 100ms. The /health endpoint is polled by Cloud Run's liveness probe — if it fails, Cloud Run automatically routes traffic to the previous revision."

6. GKE Autopilot

What is GKE Autopilot?

Google Kubernetes Engine (GKE) Autopilot is a fully managed Kubernetes mode where Google manages node provisioning, scaling, security hardening, and operating system patching. Unlike Standard GKE where you configure node pools manually, Autopilot provisions exactly the compute resources each Pod requests — billed per Pod, not per VM node.

Cluster Configuration

Cluster Name	fraudguard-cluster
Type	GKE Autopilot (fully managed)
Location	us-central1 (regional — 3 zones for HA)
Release Channel	REGULAR (stable Kubernetes versions, auto-patched)
Network	fraudguard-vpc
Subnetwork	fraudguard-subnet (10.0.0.0/16)
Pod IP Range	10.1.0.0/16 (secondary range: fraudguard-pods)
Service IP Range	10.2.0.0/20 (secondary range: fraudguard-services)
Deletion Protection	Disabled (allows terraform destroy in dev)
Terraform File	infra/terraform/gke.tf — resource google_container_cluster

What Runs Inside GKE

Namespace	Workload	Purpose
fraudguard	fraudguard-serve Deployment	FastAPI inference server (Kubernetes serving path for canary)
fraudguard	mlflow StatefulSet	MLflow tracking server with persistent volume for SQLite
fraudguard	redis Deployment	Feast feature store online layer
argocd	ArgoCD application controller	GitOps sync — watches GitHub repo, applies Helm chart changes

Kubernetes Manifests

File	Resource	Description
k8s/manifests/namespace.yaml	Namespace: fraudguard	Isolates all FraudGuard workloads
k8s/manifests/rbac.yaml	ServiceAccount: fraudguard-serve	Annotated with Workload Identity GCP SA
k8s/manifests/mlflow.yaml	Deployment + Service: mlflow	MLflow UI on port 5000

k8s/manifests/redis.yaml	Deployment + Service: redis	Redis for Feast online feature store
k8s/argocd/application.yaml	ArgoCD Application	Syncs helm/fraudguard chart from GitHub
k8s/argocd/app-of-apps.yaml	ArgoCD App-of-Apps	Root app that manages all sub-applications

Argo Rollouts — Canary Deployments

FraudGuard uses Argo Rollouts (installed in GKE) for safe canary deployments. When a new model is deployed, traffic is split progressively:

- Step 1: 10% of traffic → new Pod revision (canary)
- Step 2: Automated p99 latency gate — if >100ms, rollback; else promote
- Step 3: 100% traffic → new revision (old Pods terminated)

Why GKE Autopilot?

- **No node management:** In Standard GKE you choose VM types, node pool sizes, and manage OS patching. Autopilot removes all of that — you just define Pod resource requests.
- **Per-Pod billing:** Pay only for the vCPU and memory each Pod actually requests. If a Pod requests 0.5 CPU and 512Mi, you pay for exactly that — no wasted capacity on under-utilised VMs.
- **Security hardening by default:** Autopilot enforces Pod Security Standards, disables privileged containers, and restricts host network access automatically.
- **Regional HA:** A regional cluster spans 3 availability zones. If one zone fails, the other two keep serving traffic without manual intervention.
- **Workload Identity:** Pods can call GCP APIs (GCS, Vertex AI) using a GCP service account — no service account key files stored in Kubernetes secrets.

How to Explain GKE in an Interview

"GKE Autopilot is our Kubernetes layer. We chose Autopilot because we did not want to manage node pools — we just write Pod specs with resource requests and GKE provisions the right VMs automatically. ArgoCD watches our GitHub repo and keeps the cluster in sync with the Helm chart in helm/fraudguard/. When we merge a new image tag, ArgoCD detects the drift and applies the update. Argo Rollouts then does a canary — 10% of traffic goes to the new Pod, we check p99 latency, and only promote if the SLO holds."

7. VPC Network & Subnetwork

What is a VPC in GCP?

A Virtual Private Cloud (VPC) is a logically isolated network inside GCP. All GKE nodes, Cloud SQL instances, and internal load balancers run inside the VPC. Traffic within the VPC is free; traffic leaving the VPC to the internet incurs egress charges. VPC firewall rules control which ports and protocols are allowed.

Network Configuration

VPC Name	fraudguard-vpc
Auto-create subnets	false (we define subnets explicitly for full control)
Subnetwork Name	fraudguard-subnet
Primary CIDR	10.0.0.0/16 — GKE node IPs
Secondary CIDR (Pods)	10.1.0.0/16 — each Pod gets a /32 from this range
Secondary CIDR (Services)	10.2.0.0/20 — ClusterIP service IPs
Region	us-central1
Terraform File	infra/terraform/gke.tf — resources google_compute_network, google_compute_subnetwork

Why a Custom VPC?

- **Network isolation:** FraudGuard's GKE cluster is separated from other GCP workloads in the same project. No accidental cross-project reachability.
- **IP range control:** We choose non-overlapping CIDRs for nodes, Pods, and Services — critical if this VPC is later peered with an on-premises network or another VPC.
- **Secondary ranges for GKE:** GKE Autopilot requires secondary IP ranges for Pod and Service IPs. Without a custom subnetwork with these ranges, GKE cannot create the cluster.
- **Firewall rules:** We can add VPC firewall rules to allow only specific ports (e.g., 8000 for serving, 5000 for MLflow) and block everything else.

How to Explain VPC in an Interview

"We created a custom VPC called fraudguard-vpc instead of using the default GCP network. This gives us full control over IP ranges. The node CIDR is 10.0.0.0/16. We added two secondary ranges — 10.1.0.0/16 for Pods and 10.2.0.0/20 for Services — which GKE Autopilot requires for its internal networking. Having a custom VPC also makes it easy to add firewall rules to lock down which ports are accessible."

8. IAM & Service Accounts

What is GCP IAM?

Identity and Access Management (IAM) controls who (identity) can do what (role) on which resource (resource). In GCP every API call must be authenticated with an identity and authorised with a role. Service Accounts are non-human identities used by applications, VMs, and CI/CD pipelines to call GCP APIs.

Service Account

Account Name	fraudguard-cicd
Full Email	fraudguard-cicd@vizualflo-openclaw-prod.iam.gserviceaccount.com
Display Name	FraudGuard CI/CD Service Account
Terraform File	infra/terraform/main.tf — resource google_service_account

IAM Role Bindings

Role	Scope	Why Needed
roles/storage.objectAdmin	Project	Read/write all objects in GCS — needed for DVC push/pull, MLflow artifact upload, Terraform state read/write
roles/artifactregistry.writer	Project	Push Docker images to Artifact Registry — needed by GitHub Actions build-push job
roles/aiplatform.user	Project	Submit and monitor Vertex AI Pipeline jobs — needed by the retrain GitHub Actions job
roles/container.developer	GKE	Get cluster credentials, apply Kubernetes manifests — needed by canary-deploy job
roles/run.admin	Cloud Run	Deploy and update Cloud Run services — needed by deploy_cloud_run pipeline component
roles/iam.serviceAccountTokenCreator	SA	Allows GitHub Actions Workload Identity to impersonate this SA (see Section 9)

Kubernetes RBAC (inside GKE)

Inside the GKE cluster, the fraudguard-serve Kubernetes ServiceAccount is annotated so it can impersonate the GCP service account via Workload Identity:

k8s/manifests/rbac.yaml
apiVersion: v1
kind: ServiceAccount
metadata:
name: fraudguard-serve


```
namespace: fraudguard
```

```
annotations:
```

```
iam.gke.io/gcp-service-account: fraudguard-cicd@vizualflo-openclaw-prod.iam.gserviceaccount.com
```

Principle of Least Privilege

- The CI/CD service account has only the three roles it needs — `storage.objectAdmin`, `artifactregistry.writer`, `aiplatform.user`.
- It does NOT have `roles/editor` or `roles/owner` — it cannot delete GCP resources or modify billing.
- The GKE node service account (default Compute SA) is separate and cannot write to Artifact Registry.
- No long-lived service account keys are stored in GitHub Secrets — Workload Identity Federation is used instead (see Section 9).

How to Explain IAM in an Interview

"We follow least-privilege IAM. There is one service account — `fraudguard-cicd`. It has exactly three roles: `storage.objectAdmin` for GCS, `artifactregistry.writer` for Docker pushes, and `aiplatform.user` for Vertex AI pipeline submission. GitHub Actions authenticates as this SA using Workload Identity — no JSON key file stored anywhere. Inside GKE, the Kubernetes ServiceAccount is annotated to link to the same GCP SA via Workload Identity — so Pods can read from GCS without mounting secrets."

9. Workload Identity Federation

What is Workload Identity?

Workload Identity Federation (WIF) lets external identities — GitHub Actions runners, Kubernetes Pods, on-premises services — authenticate to GCP APIs without a service account key file (JSON key). Instead, GCP trusts a short-lived OIDC token issued by the external identity provider (GitHub's OIDC in CI, GKE's metadata server in Pods).

Two Uses in FraudGuard

Use 1 — GitHub Actions → GCP

GitHub issues a short-lived JWT to the Actions runner.
google-github-actions/auth@v2 exchanges the JWT for a GCP access token by calling
the Workload Identity Pool endpoint.
The access token impersonates fraudguard-cicd service account.
No JSON key file is stored in GitHub Secrets — only the WIF pool/provider resource IDs.

Use 2 — GKE Pod → GCP

GKE Autopilot configures each node with a metadata server (169.254.169.254).
The fraudguard-serve Pod's Kubernetes ServiceAccount is annotated:
iam.gke.io/gcp-service-account: fraudguard-cicd@...iam.gserviceaccount.com
The GKE metadata server vends a short-lived token for the linked GCP SA.
Pod code calls GCS / Vertex AI — the client library auto-fetches the token.
No secret mounting, no key rotation, no credential leakage risk.

Workload Identity Pool	fraudguard-pool (in Terraform)
Provider	fraudguard-github-provider — maps repo:thorathharish/mlops-fraud-detection-platform to the CI/CD SA
GKE Binding	google_service_account_iam_member: roles/iam.workloadIdentityUser on fraudguard-cicd
Terraform File	infra/terraform/gke.tf — resource google_service_account_iam_member

How to Explain Workload Identity in an Interview

"Workload Identity is how we avoid storing service account key files. GitHub Actions presents a short-lived OIDC token to GCP's Workload Identity endpoint, which exchanges it for a temporary access token scoped to the fraudguard-cicd SA. Inside GKE, Pods use the node metadata server — they never see a key file at all. This means there are no secrets to rotate, no risk of a key file leaking in a container image or git commit."

10. Terraform (Infrastructure-as-Code)

What is Terraform?

Terraform is an open-source Infrastructure-as-Code (IaC) tool by HashiCorp. You declare GCP resources in .tf files; Terraform computes the diff between current state and desired state, then creates, updates, or deletes resources to match. State is stored remotely in GCS so the whole team and CI/CD share the same view of infrastructure.

Terraform File Structure

File	Resources Defined
infra/terraform/main.tf	GCS data bucket, GCS TF-state backend, Artifact Registry repo, CI/CD Service Account, 3× IAM role bindings
infra/terraform/gke.tf	GKE Autopilot cluster, VPC network, Subnetwork with secondary ranges, Workload Identity IAM binding
infra/terraform/cloudrun.tf	Cloud Run v2 service with env vars, scaling config, liveness probe, public IAM binding
infra/terraform/variables.tf	project_id (default: vizualflo-openclaw-prod), region (default: us-central1)
infra/terraform/outputs.tf	Outputs: gcs_bucket_name, artifact_registry_url, cicd_service_account_email

Deployment Commands

```
# Initialise (connect Terraform to GCS backend)

terraform init -backend-config='bucket=fraudguard-tf-state'


# Preview changes

terraform plan -var='project_id=vizualflo-openclaw-prod' -var='region=us-central1'


# Apply (creates/updates all resources)

terraform apply -var='project_id=vizualflo-openclaw-prod' -var='region=us-central1' -auto-approve


# Teardown (destroy all resources — use with care!)

.\scripts\gcp_deploy.ps1 -Destroy
```

Why Terraform?

- **Reproducibility:** The entire GCP infrastructure can be recreated from scratch by running terraform apply. No clicking through the GCP console.

- **Version-controlled infrastructure:** .tf files are in git. Every change is reviewed in a PR, and you can roll back to any previous infrastructure state.
- **Remote state:** The tfstate file in GCS is the single source of truth. Two engineers can run terraform plan simultaneously and see the same state.
- **Dependency graph:** Terraform automatically orders resource creation — GCS bucket must exist before Vertex AI pipeline, VPC must exist before GKE cluster.
- **Cost control:** terraform destroy tears down all resources instantly. No forgotten VMs running overnight.

How to Explain Terraform in an Interview

"All GCP infrastructure is defined as Terraform code in infra/terraform/. We have four files: main.tf for GCS and Artifact Registry, gke.tf for the cluster and VPC, cloudrun.tf for the serving endpoint, and variables.tf for project/region. State is stored in a GCS bucket called fraudguard-tf-state so the CI/CD pipeline and any team member see the same state. To build the whole platform from zero, you run terraform apply — it creates 10+ resources in the right order automatically."

11. DVC with GCS Remote

What is DVC?

Data Version Control (DVC) is an open-source tool that adds git-like versioning to large datasets and ML models. DVC stores a small pointer file (.dvc) in git while the actual data lives in a remote storage (GCS in our case). Running `dvc pull` fetches the exact data version referenced in the current git commit.

DVC Configuration

Config File	.dvc/config
Remote Name	gcs-remote
Remote URL	gs://vizualflo-openclaw-prod-fraudguard-data/dvc
Auto-stage	true — dvc add automatically stages the .dvc pointer file in git
Auth	Application Default Credentials (gcloud auth application-default login) or Workload Identity in CI

DVC Pipeline Stages (dvc.yaml)

Stage	Command	Outputs
generate	python src/train/generate_data.py	data/raw/transactions.csv, data/raw/transactions_drifted.csv, data/reference/reference.csv
train	python src/train/train.py	MLflow run (model registered in MLflow, artifacts in GCS)
detect_drift	python src/monitoring/drift_detector.py	reports/drift_report.html, reports/drift_summary.json

How to Explain DVC in an Interview

"DVC gives us git-style versioning for datasets. The 100k-row transactions.csv is too large for git. DVC stores a SHA-256 fingerprint in a small .dvc file that we commit to git. The actual CSV lives in the dvc/ prefix of our GCS bucket. When GitHub Actions runs the retrain job, it calls dvc pull to get the exact dataset version referenced in that commit. If we change the dataset and push new data, the .dvc pointer updates in git — giving us full reproducibility: any git SHA gives you the exact dataset that produced the model."

12. GitHub Actions + GCP Integration

CI/CD Pipeline Overview

The file `.github/workflows/retrain.yml` defines an automated MLOps pipeline triggered on a cron schedule (every 6 hours) or manually. It has four sequential jobs, each using GCP services.

Job	Trigger	GCP Services Used	What Happens
detect-drift	Cron / manual	GCS (inference data), Workload Identity	Auth to GCP, pull latest.csv from GCS, run <code>drift_detector.py</code> , output <code>drift_detected=true/false</code>
retrain	<code>drift_detected == 'true'</code>	GCS (DVC remote), Vertex AI (pipeline)	<code>dvc pull</code> training data, run <code>train.py</code> , run <code>promote_model.py</code> , set <code>@production</code> alias
build-push	retrain success	Artifact Registry, Workload Identity	<code>docker build</code> , <code>docker push</code> to AR with run number tag and <code>:latest</code>
canary-deploy	build-push success	GKE Autopilot, Artifact Registry	<code>gcloud container clusters get-credentials</code> , <code>helm upgrade</code> with new image tag, Argo Rollouts canary

GitHub Secrets Required

MLFLOW_TRACKING_URI	<code>http://:5000</code> or Cloud Run MLflow URL
GCP_WORKLOAD_IDENTITY_POOL	<code>projects/{number}/locations/global/workloadIdentityPools/fraudguard-pool/providers/fraudguard-github-provider</code>
GCP_SERVICE_ACCOUNT	<code>fraudguard-cicd@vizualflo-openclaw-prod.iam.gserviceaccount.com</code>

Auth Step in Every Job

- uses: google-github-actions/auth@v2
with:
workload_identity_provider: \${{ secrets.GCP_WORKLOAD_IDENTITY_POOL }}
service_account: \${{ secrets.GCP_SERVICE_ACCOUNT }}
This exchanges the GitHub OIDC token for a GCP access token scoped to <code>fraudguard-cicd</code> .
Subsequent <code>gcloud</code> , <code>gsutil</code> , <code>docker push</code> commands all use this token automatically.

How to Explain GitHub Actions + GCP in an Interview

"Our CI/CD is GitHub Actions with four jobs. Every 6 hours, detect-drift runs — it authenticates to GCP with Workload Identity (no key file), pulls inference logs from GCS, and runs Evidently drift detection. If drift is found, the retrain job triggers: it pulls versioned training data from DVC/GCS, retrains XGBoost, and promotes the new model version. Then build-push creates a new Docker image and pushes it to Artifact Registry. Finally canary-deploy rolls it out to GKE with Argo Rollouts — 10% traffic first, then full rollout if SLO holds."

13. ArgoCD + GKE GitOps

What is GitOps?

GitOps is a pattern where the desired state of infrastructure and applications is stored in a git repository. A GitOps controller (ArgoCD) watches the repo and automatically applies changes to the cluster. The cluster always converges to match what is in git — no manual kubectl apply, no configuration drift.

ArgoCD Application Configuration

App File	k8s/argocd/application.yaml
Source Repo	https://github.com/thorathharish/mlops-fraud-detection-platform
Source Branch	main
Helm Chart Path	helm/fraudguard
Destination	in-cluster (same GKE cluster ArgoCD is installed in)
Namespace	fraudguard
Sync Policy	Automated — ArgoCD polls git every 3 minutes; on change, runs helm upgrade
App-of-Apps	k8s/argocd/app-of-apps.yaml — root ArgoCD app that manages all sub-apps

Helm Chart Structure

helm/fraudguard/
Chart.yaml — chart name: fraudguard, version: 1.0.0
values.yaml — image repo, tag, env vars (MODEL_NAME, MLFLOW_TRACKING_URI)
templates/
deployment.yaml — fraudguard-serve Deployment with liveness probe
service.yaml — ClusterIP Service on port 8000
hpa.yaml — HorizontalPodAutoscaler: min=2, max=10, CPU target=70%
rollout.yaml — Argo Rollouts canary spec: 10% -> gate -> 100%

How to Explain ArgoCD GitOps in an Interview

"ArgoCD is installed in the GKE cluster and watches our GitHub repo. When the build-push job updates values.yaml with the new image tag and merges to main, ArgoCD detects the change within 3 minutes and runs helm upgrade. Argo Rollouts takes over and performs the canary: 10% of pods get the new image, we check the p99 latency SLO, and only then promote to 100%. If the check fails, Rollouts automatically reverts to the previous revision. This means no human needs to touch kubectl for a deployment."

14. Complete GCP Resource Inventory

The following table lists every GCP resource created or used in the FraudGuard platform, from the simplest (a storage bucket) to the most advanced (GKE Autopilot with Workload Identity).

#	GCP Service	Resource Name / ID	Config Summary	Purpose
1	Cloud Storage	vizualflo-openclaw-prod-fraudguard-data	us-central1, versioned, 90-day lifecycle	Data, DVC, MLflow artifacts, inference logs, pipeline I/O
2	Cloud Storage	fraudguard-tf-state	us-central1, standard	Terraform remote state backend
3	Artifact Registry	fraudguard (Docker)	us-central1, Docker format	Private Docker image registry for fraudguard-serve
4	Vertex AI Pipelines	fraudguard-pipeline	KFP, us-central1, 4-component DAG	End-to-end ML orchestration: validate→train→promote→deploy
5	Cloud Run	fraudguard-serve	1 CPU, 1Gi, min=0, max=3, public	Serverless FastAPI inference endpoint
6	GKE Autopilot	fraudguard-cluster	us-central1, REGULAR channel, regional	Kubernetes cluster for serving, ArgoCD, Argo Rollouts
7	VPC Network	fraudguard-vpc	Custom, no auto-subnets	Network isolation for GKE cluster
8	Subnetwork	fraudguard-subnet	10.0.0.0/16, 2 secondary ranges	Node, Pod, Service IP address space for GKE
9	Service Account	fraudguard-cicd	3 IAM roles: storage, AR, Vertex AI	Non-human identity for CI/CD and Workload Identity
10	IAM Binding	roles/storage.objectAdmin	Bound to fraudguard-cicd	GCS read/write for DVC, MLflow, Terraform state
11	IAM Binding	roles/artifactregistry.writer	Bound to fraudguard-cicd	Docker image push from GitHub Actions
12	IAM Binding	roles/aiplatform.user	Bound to fraudguard-cicd	Submit Vertex AI Pipeline jobs
13	Workload Identity	fraudguard-pool / fraudguard-github-provider	GitHub OIDC mapped to fraudguard-cicd	Keyless auth from GitHub Actions to GCP
14	Workload Identity	GKE Pod annotation	iam.gke.io/gcp-service-account bound	Keyless GCS access from inside GKE Pods
15	gcloud SDK	CLI tool	Used in Vertex AI deploy component	gcloud run deploy to update Cloud Run service
16	google-cloud-storage	Python SDK 3.1.0	google.cloud.storage.Client()	Vertex AI validate_data component downloads from GCS
17	google-cloud-aiplatform	Python SDK 1.95.1	aiplatform.PipelineJob	Submit and monitor Vertex AI pipeline from Python

18	dvc-gs	DVC GCS remote plugin >=3.0.0	gsutil under the hood	Push/pull training data between local and GCS
----	--------	----------------------------------	-----------------------	--

15. How to Explain This Project (Interview Script)

Use these structured answers when explaining the project in an interview or demo. Each answer is 2–4 sentences, covering what, why, and how.

Q: What is FraudGuard in one sentence?

FraudGuard is a production-grade MLOps pipeline on GCP that trains an XGBoost fraud detection model, serves it via Cloud Run, automatically detects data drift using Evidently, and triggers retraining through a Vertex AI Pipeline when drift exceeds 30% — forming a closed loop with zero manual intervention.

Q: Which GCP services did you use and why?

GCS for all persistent storage (training data via DVC, model artifacts via MLflow, pipeline I/O). Artifact Registry for private Docker images — no Docker Hub rate limits and native GKE/Cloud Run integration. Vertex AI Pipelines to orchestrate the 4-step train→promote→deploy DAG. Cloud Run for serverless inference — scales to zero overnight saving cost. GKE Autopilot for the Kubernetes layer where ArgoCD does GitOps and Argo Rollouts does canary deployments.

Q: How does the automated retraining work?

GitHub Actions runs every 6 hours, authenticates to GCP via Workload Identity (no key file), pulls the latest prediction logs from GCS, and runs Evidently drift detection. If more than 30% of features drift — amount, velocity, distance, hour, international flag — the detector exits with code 1. GitHub Actions interprets that as `drift_detected=true` and triggers the retrain job, which calls `dvc pull` to get the versioned dataset from GCS, retrains XGBoost, and promotes the new model version to the production alias in MLflow.

Q: How did you handle security? No hardcoded secrets?

We used Workload Identity Federation throughout. GitHub Actions presents its OIDC token to GCP's WIF endpoint, which exchanges it for a scoped access token — no JSON key file stored in GitHub Secrets. Inside GKE, the `fraudguard-serve` Kubernetes ServiceAccount is annotated with the GCP service account email; the node metadata server vends short-lived tokens automatically. The service account has only the three roles it needs: `storage.objectAdmin`, `artifactregistry.writer`, and `aiplatform.user`.

Q: What is the serving architecture?

There are two serving paths. The Cloud Run path is serverless: the FastAPI container is deployed to Cloud Run (min=0, max=3 instances, 1 CPU, 1Gi), and predictions are made via `POST /predict`. The Kubernetes path deploys the same image to GKE Autopilot via ArgoCD GitOps; Argo Rollouts does a 10% canary, checks p99 latency SLO (<100ms), then promotes to 100%. Prometheus scrapes `/metrics` on the FastAPI container; Grafana visualises fraud scores, latency, and error rates.

Q: How does Terraform fit in?

All GCP resources — GCS buckets, Artifact Registry repo, GKE cluster, VPC, Cloud Run service, and service account IAM bindings — are defined as Terraform code in `infra/terraform/`. Terraform state is stored in a GCS bucket so it is shared across the team and CI/CD. Running `terraform apply` recreates the entire GCP infrastructure from zero. The GKE cluster has `deletion_protection` disabled so we can `terraform destroy` it after demos to avoid idle charges.

Q: What happens if a newly trained model is worse than the current one?

The `promote_model` Vertex AI component checks that the new model's PR-AUC is at least 0.85 before setting the `@production` alias. If the threshold is not met, the component exits without updating the alias — the existing production model keeps serving. On the Kubernetes path, Argo Rollouts has an automated p99 latency analysis step: if p99 exceeds 100ms during the 10% canary phase, Rollouts automatically reverts to the previous revision.

Q: How is the training dataset versioned?

We use DVC (Data Version Control) with a GCS remote. The 100k-row `transactions.csv` is stored in `gs://...fraudguard-data/dvc/` as a SHA-256-addressed blob. In git we only commit the small `transactions.csv.dvc` pointer file. When GitHub Actions runs the retrain job, it calls `dvc pull` to download the exact dataset version referenced in the current commit — ensuring that any git SHA gives you the exact data that produced the model registered in MLflow at that point.

FraudGuard MLOps Platform

GCP Architecture Reference | vizualflo-openclaw-prod |
us-central1

Prepared by Harish Thorath — June 2026

All infrastructure defined as Terraform IaC in infra/terraform/